

Optimization 1

ISE 5406

Lecture 1

Dr. Robert Hildebrand

Grado Department of Industrial & Systems Engineering
Virginia Tech

Reading for this lecture

- Chapter 1 and Section 2.1 of Nokedal and Wright (2006)
- Chapter 1 of Bazaraa, Sherali, and Shetty (2006)
- Chapter 1 of Boyd and Vandenberghe

About Your Instructor

Dr. Robert Hildebrand

- Assistant Professor, Grado Department of ISE, Virginia Tech
- Office: Durham 223
- Email: rhil@vt.edu

Background

- Ph.D. in Mathematics, UC Davis (2013)
- Postdoc at ETH Zürich and IBM Research
- Research: Integer programming, combinatorial optimization, lattice-based methods

Office Hours

- Tuesdays 9:15 AM – 10:45 AM
- Thursdays 11:00 AM – 12:30 PM
- Or by appointment

I encourage you to come to office hours! It's a great way to get help and discuss the material.

Mathematical Formulation or Modeling

Optimization is the maximization or minimization of an **objective** subject to **constraints** on its **variables**.

- **Objective:** e.g., profit, time, cost, etc.
 - **Variables (or unknowns):** e.g., production/inventory level, etc.
 - **Constraints:** non-negativity, bounds, physical laws, etc.
1. Simplistic model $\Rightarrow$ no insight into the true problem.
 2. Complex model $\Rightarrow$ may be difficult to solve.

Good modeling is half the solution! (see Section 1.3 of Bazaraa et al.)

Modeling reference: Fourer, Gay, and Kernighan (2003),
AMPL: A Modeling Language for Mathematical Programming,
<https://vanderbei.princeton.edu/307/textbook/AMPLbook.pdf>

A Problem for the Ages

Find the ages of Justin and Brad, given that the sum of their ages equals 76 years and that Brad will be twice as old as Justin in 7 years' time.

Mathematical Modeling

A Problem for the Ages

Find the ages of Justin and Brad, given that the sum of their ages equals 76 years and that Brad will be twice as old as Justin in 7 years' time.

Solution: Let x and y denote the ages of Justin and Brad, respectively. Then

$$x + y = 76$$

$$y + 7 = 2(x + 7)$$

Upon solving these equations, we get $x = 23$ and $y = 53$.

Mathematical Modeling

A Problem for the Ages

Find the ages of Justin and Brad, given that the sum of their ages equals 76 years and that Brad will be twice as old as Justin in 7 years' time.

Solution: Let x and y denote the ages of Justin and Brad, respectively. Then

$$x + y = 76$$

$$y + 7 = 2(x + 7)$$

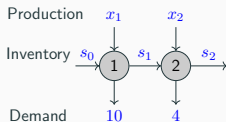
Upon solving these equations, we get $x = 23$ and $y = 53$.

Geometric Interpretation: Intersection of two lines.

Mathematical Optimization

Production Planning Problem

In a manufacturing firm, given the demand for two days along with per-unit production and inventory costs, decide the production schedule for both days.



Decision Variables

x_1 : Production on Day 1

x_2 : Production on Day 2

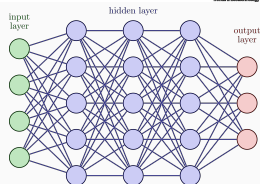
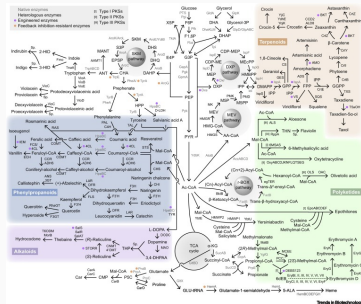
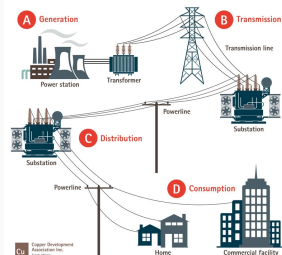
s_0 : Initial Inventory

s_1 : Inventory at the end of Day 1

s_2 : Inventory at the end of Day 2

Data: Per-unit production cost on Day 1 and Day 2: \$10 and \$16.
Per-unit inventory cost = \$5.

The Universe is Nonlinear



Applications of Nonlinear Programming

Industrial & Systems Engineering (ISE)

- Supply chain network design, facility location
- Production planning with nonlinear costs
- Portfolio optimization (Markowitz mean-variance)
- Queueing system optimization

Computer Science (CS)

- Algorithm design and analysis
- Computational geometry (e.g., smallest enclosing ball)
- Graphics and computer vision (bundle adjustment, mesh optimization)

Machine Learning / AI

- Linear and logistic regression, support vector machines
- Neural network training (deep learning, LLMs)
- Reinforcement learning policy optimization

Applications of Nonlinear Programming (cont.)

Electrical & Computer Engineering (ECE)

- Signal processing and filter design
- Power systems optimal power flow (OPF)
- Circuit design and VLSI optimization
- Communications: beamforming, channel capacity

Chemical Engineering

- Process design and experiment design
- Reaction kinetics parameter estimation
- Thermodynamic equilibrium calculations

Mathematics & Physics

- Sphere packing (Kepler conjecture, error-correcting codes)
- Minimal surfaces and geometric optimization
- Molecular geometry optimization
- Protein folding

Application: Pooling Problem (Chemical/Petroleum)

Problem: Blend raw materials through intermediate pools to create products meeting quality specifications at minimum cost.

Variables:

- x_{ij} : amount of raw material i sent to pool j
- y_{jk} : amount from pool j to product k

Why Nonlinear? Quality of pool output depends on the *ratio* of inputs:

$$\text{Quality of pool } j = \frac{\sum_i q_i x_{ij}}{\sum_i x_{ij}} \quad (\text{bilinear!})$$

Applications:

- Petroleum refining: blending crude oils for fuel properties
- Food industry: mixing ingredients for nutritional targets
- Water treatment: combining sources to meet standards

Application: Unit Commitment (Power Systems)

Problem: Decide which power generators to turn on/off and their output levels over 24 hours to meet demand at minimum cost.

Variables:

- $u_{gt} \in \{0, 1\}$: is generator g on at time t ?
- $p_{gt} \in \mathbb{R}$: power output of generator g at time t

Why Nonlinear? Fuel cost is typically quadratic:

$$\text{FuelCost}(p_{gt}) = a_g + b_g p_{gt} + c_g p_{gt}^2$$

Constraints:

- Demand: $\sum_g p_{gt} = \text{Demand}_t$
- Generation limits: $P_g^{\min} u_{gt} \leq p_{gt} \leq P_g^{\max} u_{gt}$
- Ramping, minimum up/down times

Application: AC Optimal Power Flow

Problem: Optimize power generation and transmission to minimize cost while satisfying physical laws of electricity.

Why Nonlinear? Power flow equations involve products of voltages and trigonometric functions:

$$P_i = \sum_j V_i V_j (G_{ij} \cos(\theta_i - \theta_j) + B_{ij} \sin(\theta_i - \theta_j))$$

Key Challenges:

- Nonconvex constraints from AC power flow physics
- Large-scale: thousands of buses and transmission lines
- Must solve quickly for real-time grid operation

Impact: Efficient OPF solutions save billions of dollars annually in electricity costs and are critical for grid reliability.

Application: Portfolio Optimization (Finance)

Markowitz Mean-Variance Model: Find portfolio weights minimizing risk for a target return.

$$\min_{\mathbf{x}} \mathbf{x}^T Q \mathbf{x} \quad \text{s.t.} \quad \sum_i x_i = 1, \quad \sum_i \mu_i x_i \geq \rho, \quad \mathbf{x} \geq 0$$

- x_i : fraction of portfolio in asset i
- Q : covariance matrix of returns (positive semidefinite)
- μ_i : expected return of asset i ; ρ : target return

Extensions with Integer Variables:

- Buy-in thresholds: $\ell_i z_i \leq x_i \leq u_i z_i, \quad z_i \in \{0, 1\}$
- Cardinality constraints: invest in at most k assets
- Transaction costs, sector constraints

Higher Moments: Modern models include skewness (prefer positive) and kurtosis (prefer low) — leads to polynomial optimization!

Application: Machine Learning with Sparsity

Sparse Regression (ℓ_0 -regularization):

$$\min_{\mathbf{x} \in \mathbb{R}^n} \|\mathbf{y} - A\mathbf{x}\|_2^2 + \mu \|\mathbf{x}\|_0$$

where $\|\mathbf{x}\|_0$ counts nonzero entries (not a norm!).

Why Sparse? Interpretability, feature selection, avoiding overfitting.

Sparse PCA:

$$\max_{\mathbf{x}} \mathbf{x}^T \Sigma \mathbf{x} \quad \text{s.t.} \quad \|\mathbf{x}\|_2 \leq 1, \|\mathbf{x}\|_0 \leq k$$

Find principal components using at most k features.

The Challenge: $\|\mathbf{x}\|_0$ is discontinuous and nonconvex.

Common approaches: ℓ_1 relaxation (LASSO), mixed-integer formulations, greedy algorithms.

Application: Distance Geometry & Molecular Structure

Problem: Given distances between some pairs of atoms, determine 3D positions.

$$\min_{\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^3} \sum_{\{i,j\} \in E} (\|\mathbf{x}_i - \mathbf{x}_j\|^2 - d_{ij}^2)^2$$

Applications:

- Protein structure determination from NMR data
- Sensor network localization
- Robotics and kinematics

Kissing Number Problem: How many unit spheres can touch a central unit sphere?

- $K = 2$: answer is 6
- $K = 3$: answer is 12 (Newton vs. Gregory debate!)
- $K = 4$: answer is 24
- $K = 8, 24$: known; other dimensions still open!

This is a nonconvex feasibility problem with deep connections to coding theory.

Application: Robust Optimization

Problem: Make decisions that perform well under uncertainty.

$$\min_{\mathbf{y} \in Y} \max_{\mathbf{a} \in \mathcal{U}} \mathbf{a}^T \mathbf{y}$$

Uncertainty Set $\mathcal{U}$:

- Ellipsoidal: $\mathcal{U} = \{\mathbf{a} : \|\mathbf{a} - \tilde{\mathbf{a}}\|_2 \leq \epsilon\}$
- Polyhedral: $\mathcal{U} = \{\mathbf{a} : A\mathbf{a} \leq b\}$
- Cardinality-constrained: only k parameters can deviate

Why Nonlinear?

- Inner max over ellipsoid leads to second-order cone constraints
- Cardinality constraints introduce combinatorial structure
- Products of uncertain parameters and decisions

Applications: Supply chain under demand uncertainty, portfolio optimization with uncertain returns, engineering design with manufacturing tolerances.



PROBLEM SOLVING

Mathematical Optimization: Numerical Algorithms

- There is no *universal* optimization algorithm.
- There are numerous algorithms, each of which is tailored to a particular type of optimization problem.

1. Deterministic Optimization

- Linear Programming [ISE 5405]
- **Nonlinear Programming** [ISE 5406]
- Integer Programming [ISE 6414]
- Conic Programming [ISE 6514]
- Mixed-Integer Nonlinear Programming [ISE 6984]
- $\vdots$

2. Optimization Under Uncertainty

- Stochastic and Robust Optimization [ISE 6454]
- $\vdots$

Mathematical Optimization: Optimality Conditions

- After an optimization algorithm has been applied to the model, we must be able to recognize whether it has succeeded in finding an optimal solution.
- In many cases, there are elegant mathematical expressions, known as **Optimality Conditions**, for checking that the returned solution is indeed an optimal solution of the problem.
- If these optimality conditions are not satisfied, they may provide useful information on how the current estimate of the solution can be improved.
- Optimality conditions are related to **Sensitivity Analysis**, which studies the sensitivity of the solution to changes in model parameters.

Mathematical Optimization

- **Definition:** Optimization is the maximization or minimization of a function subject to constraints on its variables.
- Mathematically, an optimization problem can be written as:

$$\begin{aligned} &\text{Minimize } f(\mathbf{x}) && // \text{ objective function} \\ &\text{subject to } g_i(\mathbf{x}) \leq 0, \quad i \in \mathcal{I}, && // \text{ inequality constraints} \\ &\quad \quad \quad h_i(\mathbf{x}) = 0, \quad i \in \mathcal{E}, && // \text{ equality constraints} \\ &\quad \quad \quad \mathbf{x} \in X \subset \mathbb{R}^n, \end{aligned}$$

where f , g_i , and h_i are scalar-valued functions of variables $\mathbf{x}$, and $\mathcal{I}$, $\mathcal{E}$ are finite sets of indices.

- Depending on (f, g, h, X) , the problem is an LP, mixed-integer linear program, convex program, nonlinear program (NLP), MINLP, stochastic program, bilevel program, etc.

Why is Nonlinear Programming Harder than LP?

In **linear programming**, optimal solutions are “nice”:

- Solutions are rational (if data is rational)
- Solutions exist or problem is unbounded/infeasible
- Polynomial-time algorithms exist (e.g., interior point)

In **nonlinear programming**, new challenges arise:

Example 1: Irrational Solutions

$$\min x \quad \text{s.t.} \quad x^2 \geq 2$$

Optimal solution: $x^* = -\sqrt{2}$ (irrational — cannot be represented exactly!)

Example 2: Infimum Not Attained

$$\min y \quad \text{s.t.} \quad xy \geq 1, x > 0, y > 0$$

Infimum is 0, but **no feasible point achieves it!**

We need new mathematical tools (e.g., infimum vs. minimum, closed sets).

Why is Nonlinear Programming Harder than LP? (cont.)

Example 3: Hidden Discrete Structure

$$\min x + y \quad \text{s.t.} \quad x(1 - x) = 0, \quad A \begin{pmatrix} x \\ y \end{pmatrix} \leq b$$

The constraint $x(1 - x) = 0$ forces $x \in \{0, 1\}$.

⇒ Binary/integer constraints can be encoded as polynomial constraints!

This means NLP is *at least as hard* as integer programming.

Complexity Result:

- Deciding feasibility of polynomial systems is **undecidable** in general (Hilbert's 10th problem over $\mathbb{Z}$).
- Even over $\mathbb{R}$, polynomial optimization is **NP-hard** and checking local optimality can be NP-hard.

The Two Faces of Nonlinear Programming

Structure matters!

- **Bad news:** Some NLPs with only a **few variables** are impossible to solve exactly.
 - General polynomial optimization
 - Nonconvex quadratic programming
- **Good news:** Some NLPs with **millions of variables** can be solved efficiently!
 - Convex optimization (linear, quadratic, semidefinite programming)
 - Neural network training via stochastic gradient descent

The **type** of nonlinear program determines whether it is tractable or intractable — not just the size!

What We Will Learn in This Course

In this course, we will develop:

1. Optimality Conditions (Theory)

- How do we *recognize* an optimal solution?
- Fritz John conditions, Karush-Kuhn-Tucker (KKT) conditions
- Constraint qualifications and when conditions are sufficient

2. Iterative Algorithms

- Gradient descent, Newton's method, conjugate gradient
- Projected gradient, proximal methods, interior point methods
- **Convergence**: Does the algorithm find a solution?
- **Rate of convergence**: How fast does it get there?

3. Mathematical Foundations

- Convex analysis: convex sets, convex functions, subgradients
- Real analysis: limits, continuity, compactness, closure
- Linear algebra: gradients, Hessians, positive definiteness

What We Will Learn in This Course (cont.)

4. Computational Skills

- Implementing optimization algorithms in code (Python)
- Using optimization solvers for real-world problems
- Understanding algorithm behavior through experiments

5. Applications

- Machine learning: linear regression, classification, neural networks/LLMs
- Portfolio optimization, engineering design
- Connecting theory to practice

Course Philosophy:

Understand the *theory* deeply so you can
apply algorithms *intelligently* and debug them when they fail.

Coding in This Course

Programming Level

- We will mostly stay at “Jupyter-level coding” — not overly complex
- Focus on implementing algorithms and running experiments, not software engineering
- I code in **Python**; if you prefer another language, that's mostly fine, but it may be harder to follow my examples

Optimization Solvers We May Use

- **CVXPY / CVX**: Convex optimization modeling
- **SciPy**: Suite of algorithms for testing and comparison (`scipy.optimize`)
- **IPOPT**: Interior point optimizer for large-scale NLP
- **Gurobi / Mosek**: Commercial solvers (free academic licenses)
- **BARON**: Global optimization for nonconvex problems
- **SeDuMi**: Semidefinite programming

Don't worry if you haven't used these before — we'll learn as we go!

Methodological Focus of This Course

Our Focus: Continuous Nonlinear Programming

- **Optimality theory:** KKT conditions, constraint qualifications, duality
- **Iterative algorithms:** Gradient descent, Newton, conjugate gradient, interior point
- **Convergence analysis:** Does the algorithm converge? How fast?
- **Convexity:** When structure guarantees global optimality

Goal: Find **local optima** efficiently for smooth, continuous problems.

Often sufficient for convex problems (local = global) and practical for many applications.

What We Won't Cover (Much):

The MINLP world seeking **exact/global solutions** to hard nonconvex problems uses a different toolkit:

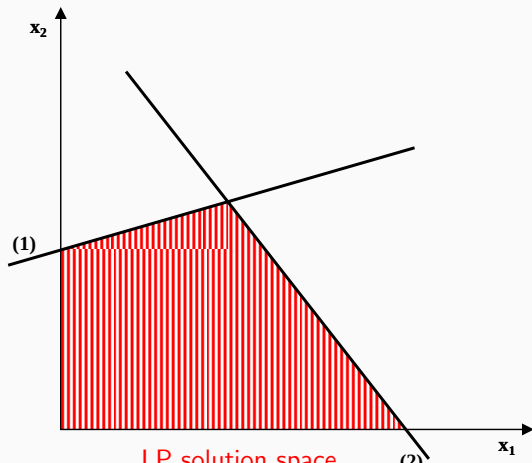
- Spatial branch-and-bound, convexification techniques
- LP, QP, and SDP relaxations
- Reformulations, liftings, extended formulations
- Heuristics and problem-specific structural results

These are covered in ISE 6414 (Integer Programming) and ISE 6984 (MINLP).

Mathematical Optimization

Linear Program (LP):

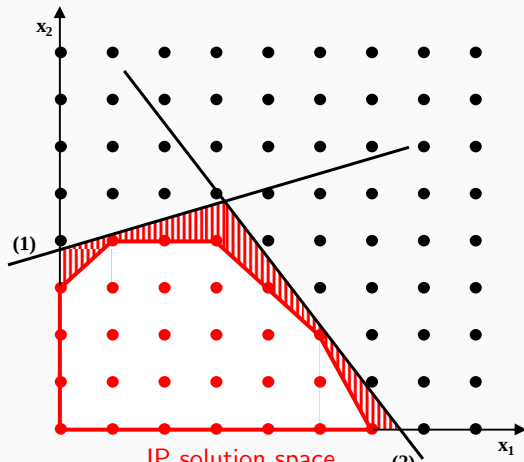
$$\max \{cx : Ax \leq b, x \in \mathbb{R}^n\}$$



Mathematical Optimization

Integer Linear Program (ILP):

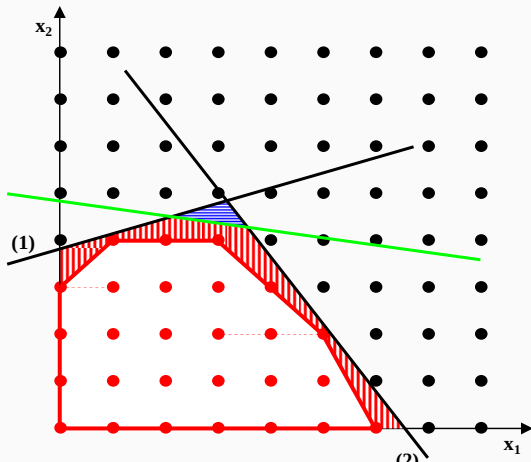
$$\max \{cx : Ax \leq b, x \text{ integral}\}$$



Mathematical Optimization

Integer Linear Program (ILP):

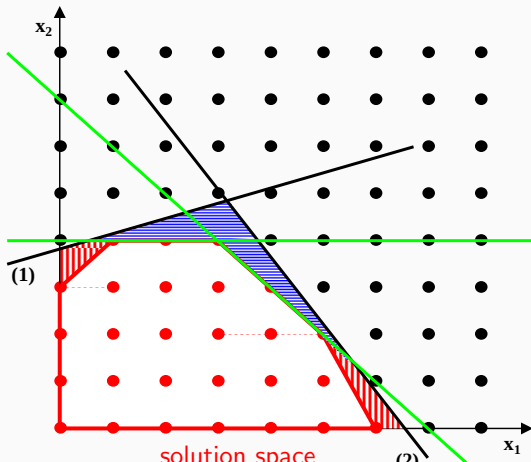
$$\max \{cx : Ax \leq b, x \text{ integral}\}$$



- **Cutting Plane Algorithm**
- **Cutting planes:** Cut off part of relaxed LP region without eliminating ILP solutions

Integer Linear Program (ILP):

$$\max \{cx : Ax \leq b, x \text{ integral}\}$$



- **Cutting Plane Algorithm**
- **Cutting planes:** Cut off part of relaxed LP region without eliminating ILP solutions
- **Strongest Cuts: Facets**

Nonlinear Programming (NLP)

$$\min (x_1 - 2)^2 + (x_2 - 1)^2$$

$$s.t. \ x_1^2 - x_2 \leq 0,$$

$$x_1 + x_2 \leq 2.$$

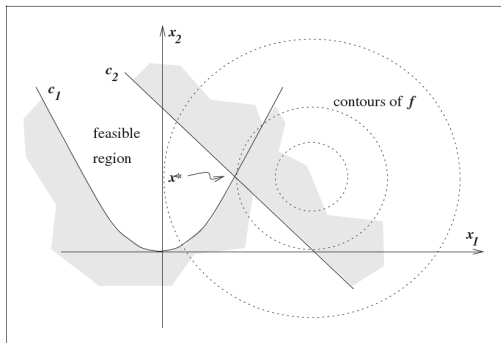


Figure 1.1 Geometrical representation of an optimization problem.

Constrained Optimization Problem:

$$\min (x_1 - 2)^2 + (x_2 - 1)^2$$

$$s.t. \ x_1^2 - x_2 \leq 0,$$

$$x_1 + x_2 \leq 2.$$

Nonlinear Programming (NLP)

Constrained Optimization Problem:

$$\min (x_1 - 2)^2 + (x_2 - 1)^2$$

$$s.t. \ x_1^2 - x_2 \leq 0,$$

$$x_1 + x_2 \leq 2.$$

Unconstrained Optimization Problem:

$$\min x_1 + x_1x_2 + x_2^2$$

$$s.t. \ x \in \mathbb{R}^2.$$

Nonlinear Programming: Facility Location Problem

- Suppose that there are n markets with known demands and locations. These demands are to be met from m warehouses of known capacities.
- The problem is to determine the locations of the warehouses so that the total distance weighted by the shipment from the warehouses to the markets is minimized.

Problem Data (Parameters):

1. c_i : capacity of warehouse i for $i = 1, \dots, m$
2. (a_j, b_j) : known location of market j for $j = 1, \dots, n$
3. r_j : known demand at market j for $j = 1, \dots, n$

Decision Variables:

1. (x_i, y_i) : unknown location of warehouse i for $i = 1, \dots, m$.
2. w_{ij} and d_{ij} : units shipped and distance, respectively, from warehouse i to market area j for $i = 1, \dots, m$; $j = 1, \dots, n$

Objective Function: Minimize $\sum_{i=1}^m \sum_{j=1}^n w_{ij} d_{ij}$

Constraints:

1. Known capacity of warehouse i :

$$\sum_{j=1}^n w_{ij} \leq c_i, \quad \text{for } i = 1, \dots, m$$

2. Known demand of market j :

$$\sum_{i=1}^m w_{ij} = r_j, \quad \text{for } j = 1, \dots, n$$

3. Non-negativity constraints: $w_{ij} \geq 0$ for all i, j .

Decision Variables:

1. (x_i, y_i) : unknown location of warehouse i for $i = 1, \dots, m$.
2. w_{ij} and d_{ij} : units shipped and distance, respectively, from warehouse i to market area j for $i = 1, \dots, m$; $j = 1, \dots, n$

Objective Function: Minimize $\sum_{i=1}^m \sum_{j=1}^n w_{ij} d_{ij}$

- Rectilinear Distance: $d_{ij} = |x_i - a_j| + |y_i - b_j|$
- Euclidean Distance: $d_{ij} = \sqrt{(x_i - a_j)^2 + (y_i - b_j)^2}$
- l_p Norm Metrics: $d_{ij} = [|x_i - a_j|^p + |y_i - b_j|^p]^{1/p}$

Decision Variables:

1. (x_i, y_i) : unknown location of warehouse i for $i = 1, \dots, m$.
2. w_{ij} and d_{ij} : units shipped and distance, respectively, from warehouse i to market area j for $i = 1, \dots, m$; $j = 1, \dots, n$

Objective Function: Minimize $\sum_{i=1}^m \sum_{j=1}^n w_{ij} d_{ij}$

- Rectilinear Distance: $d_{ij} = |x_i - a_j| + |y_i - b_j|$
- Euclidean Distance: $d_{ij} = \sqrt{(x_i - a_j)^2 + (y_i - b_j)^2}$
- l_p Norm Metrics: $d_{ij} = [|x_i - a_j|^p + |y_i - b_j|^p]^{1/p}$

Note: If the locations of the warehouses are fixed, then the d_{ij} values are known and this problem reduces to a linear program known as the **transportation problem**.

Decision Variables:

1. (x_i, y_i) : unknown location of warehouse i for $i = 1, \dots, m$.
2. w_{ij} and d_{ij} : units shipped and distance, respectively, from warehouse i to market area j for $i = 1, \dots, m$; $j = 1, \dots, n$

Objective Function: Minimize $\sum_{i=1}^m \sum_{j=1}^n w_{ij} d_{ij}$

- Rectilinear Distance: $d_{ij} = |x_i - a_j| + |y_i - b_j|$
- Euclidean Distance: $d_{ij} = \sqrt{(x_i - a_j)^2 + (y_i - b_j)^2}$
- l_p Norm Metrics: $d_{ij} = [|x_i - a_j|^p + |y_i - b_j|^p]^{1/p}$

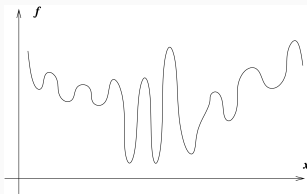
Note: If the locations of the warehouses are fixed, then the d_{ij} values are known and this problem reduces to a linear program known as the **transportation problem**.

See Chapter 1 of Bazaraa et al. for more examples of NLP models.

Fundamentals of Unconstrained Optimization

Minimize $f(\mathbf{x})$, where $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is smooth
 $x \in \mathbb{R}^n$

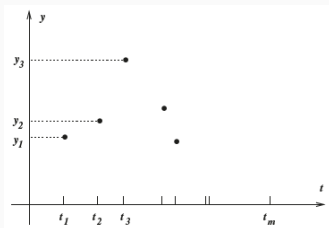
- Usually, we lack global perspective on the function f .



- All we know are the values of f and maybe some of its derivatives at a set of points $\mathbf{x}_0, \mathbf{x}_1, \mathbf{x}_2, \dots$
- We wish to design robust algorithms that solve the problem without using too much computer time or storage.
- Often, the information about f or its derivative is not cheap, so we prefer algorithms that do not call for this information unnecessarily.

Example: Least-Squares Problem

- Given the measurements $y_1, y_2, \dots, y_m$ of a signal taken at times $t_1, t_2, \dots, t_m$, find a curve that fits this data.



- Suppose we model the relationship between y and t as:

$$\phi(t; \mathbf{x}) = x_1 + x_2 e^{-(x_3 - t)^2 / x_4} + x_5 \cos(x_6 t)$$

where real numbers x_i , $i = 1, \dots, 6$, are model parameters.

Example: Least-Squares Problem

- Choose $x_1, x_2, \dots, x_6$ so that the model values $\phi(t_j; \mathbf{x})$ fit the observed data y_j , $j = 1, \dots, m$, as closely as possible.
- **Variables:** $\mathbf{x} = (x_1, \dots, x_6) \in \mathbb{R}^6$
- **Objective function:** Minimize the residuals which measure the discrepancy between the model and the observed data, e.g.,

$$\min f(\mathbf{x}) = r_1^2(\mathbf{x}) + \dots + r_m^2(\mathbf{x})$$

where $r_j(\mathbf{x}) = y_j - \phi(t_j, \mathbf{x})$ for $j = 1, \dots, m$.

- Chapter 10 of Nocedal and Wright specializes algorithms for unconstrained optimization to least-squares problems.

Unconstrained Optimization: Local and Global Minimizers

Global Minimizer: A point $\mathbf{x}^*$ is a global minimizer if

$$f(\mathbf{x}^*) \leq f(\mathbf{x}) \text{ for all } \mathbf{x}.$$

Local Minimizer: A point $\mathbf{x}^*$ is a local minimizer if there is some neighborhood $\mathcal{N}$ of $\mathbf{x}^*$ such that

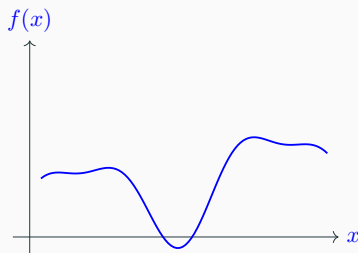
$$f(\mathbf{x}^*) \leq f(\mathbf{x}) \text{ for all } \mathbf{x} \in \mathcal{N}.$$

Strict Local Minimizer: $f(\mathbf{x}^*) < f(\mathbf{x})$

for all $\mathbf{x} \in \mathcal{N}$, $\mathbf{x} \neq \mathbf{x}^*$.

Note: A neighborhood of $\mathbf{x}^*$ is an open set containing $\mathbf{x}^*$. An

ϵ -neighborhood: $\mathcal{N}_\epsilon(\mathbf{x}) := \{\mathbf{y} : \|\mathbf{y} - \mathbf{x}\|_2 < \epsilon\}.$



Isolated Local Minimizer: A point $\mathbf{x}^*$ is an isolated local minimizer if there is some neighborhood $\mathcal{N}$ of $\mathbf{x}^*$ such that $\mathbf{x}^*$ is the only local minimizer in $\mathcal{N}$.

- Note: Strict local minimizers are not always isolated, but all isolated local minimizers are strict. For instance, the point $x^* = 0$ is a strict (but not isolated) local minimizer of the function

$$f(x) = \begin{cases} x^4 \cos(1/x) + 2x^4 & \text{if } x \neq 0, \\ 0 & \text{if } x = 0. \end{cases}$$

Plot this function and see how it looks near $x = 0$!

(See Figure 3.7 of Bazaraa et al. for other examples.)

Course Structure and Grading

Grading Breakdown

- **Homework (50%):** 5 assignments, approximately bi-weekly
- **Quizzes (10%):** Online via Canvas, bi-weekly (alternating with HW)
- **Exams (40%):** Three in-class exams
 - Exam 1 (12%): Thursday, March 5
 - Exam 2 (12%): Thursday, April 9
 - Final Exam (16%): Monday, May 11, 2:05–4:05 PM

Important: Please note the exam dates!

You must be present for in-class exams.

If you have a conflict, notify me **immediately**.

What to Expect

Quizzes — Online, bi-weekly

- Designed to keep you current with the material
- Cover key definitions, theorems, and algorithms
- Relatively straightforward if you attend class and do the reading

Homework — Bi-weekly, $\text{\LaTeX}$ required

- **Challenging!** Requires deep engagement with the material
- 3–6 graded problems per assignment
- Collaboration encouraged, but write up solutions independently

Exams — In-class, closed-book

- Medium difficulty — between quizzes and homework
- One cheat sheet allowed (two for the final)
- Tests understanding of concepts and ability to apply them

Homework Late Policy

Days Late	Penalty
1 day	20%
2 days	50%
>2 days	100% (no credit)

- Homework is due on Fridays at **12:01 PM**
- Even being one minute late counts as a full day late
- **Start early!** These assignments take time.

Extenuating Circumstances

- If you have illness, family emergency, or other serious issues, contact me **as soon as possible**
- I am willing to work with you, but communication is key

AI Policy for This Course

The Challenge

- AI tools (ChatGPT, Claude, Copilot, etc.) can likely solve most homework problems instantly
- This creates a teaching challenge: how do you learn if AI does everything?

My Approach

- **Prohibited:** Using AI to solve homework or exam problems
- **Encouraged:** Using AI to help with $\LaTeX$, study for exams, learn concepts
- In-class exams serve as a check on your understanding

My Recommendation

- Do homework **mostly on your own** first
- **Review** your solutions with AI before exams — great for studying!
- **But remember:** It takes an expert to verify if AI output is correct!

Note: This policy may evolve as we learn more about effective AI use in education.

Your Well-Being Matters

There are many stressful things happening in the world right now.

I want to be sensitive to this in how I approach homework, exams, and your studies.

If life gets complicated:

- Please feel free to reach out to me
- I am happy to work with you to accommodate your needs
- Communication is key — let me know early if you're struggling

Your success in this course matters to me,
but so does your well-being.

Accommodations (SSD)

If you require accommodations through **Services for Students with Disabilities (SSD)**:

- Please discuss your needs with me — either in office hours or via email
- Please do this within the first 3 weeks of the semester
- This gives us time to set up appropriate accommodations for homework and exams
- If you need course materials in a specific ADA-accessible format, please let me know

To request accommodations:

- Obtain a faculty letter from the SSD office
- Bring it to me so we can discuss how to best support you

More information: <https://www.ssd.vt.edu>

A Note About Canvas

I will sometimes forget to push certain buttons on Canvas!

If you notice that something on the Canvas course website is:

- Missing
- Unpublished (you can't access it)
- Incorrect
- Confusing

Please let me know!

I appreciate your patience and your help in keeping the course running smoothly.

Summary and Next Steps

Reading for the next lecture

- Chapter 2 of Bazaraa et al. (2006)
- Sections 2.1 to 2.3 and Sections 3.1 to 3.2 of Boyd and Vandenberghe

Next Time:

- Convex sets and their properties
- Linear algebra fundamentals
- Building the mathematical foundation for optimization

Questions?